Reproducing *Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles*

Theodoris Donval
EURECOM
theodoris.donval@eurecom.fr

Lucas Aliome
EURECOM
lucas.aliome@eurecom.fr

Abstract

We reproduce the regression experiments of Lakshminarayanan et al. (NeurIPS 2017), which introduces *Deep Ensembles*, a simple non-Bayesian recipe for predictive uncertainty. The recipe has three parts: train a probabilistic network with a proper scoring rule, ensemble M such networks, and optionally add adversarial training. With a small PyTorch re-implementation we reproduce the 1-D toy regression of Figure 1 and the k-fold RMSE/NLL evaluation protocol of Table 1, together with the single-network-versus-ensemble ablation. On the toy task we recover the paper's main qualitative result: the predictive uncertainty only grows away from the training data once the networks are ensembled. The ablation confirms that ensembling lowers the held-out NLL. We also document a concrete reproduction issue. The learning rate stated in the paper (0.1) was unstable in our runs, and we had to lower it to 0.01 to recover the reported behaviour. All code, notebooks and results are available at `https://github.com/Theodoris-D/deep-ensembles-reproduction`.

1 Summary of the paper

Problem. Standard neural networks are usually trained as point predictors, and they are known to be *overconfident*: they return a single output with no reliable measure of how uncertain it is. This overconfidence tends to be worst on inputs that lie far from the training distribution, which is often exactly where a trustworthy uncertainty estimate matters most. Quantifying predictive uncertainty is therefore important for safety-critical and decision-making applications. The main principled approach at the time, Bayesian neural networks, relies on approximate inference that is often complex, sensitive to the choice of prior, and hard to scale. The paper asks whether a simpler, non-Bayesian alternative can match or beat these methods.

Method. The recipe has three ingredients. *(1) A proper scoring rule as the training criterion.* Rather than producing only a point estimate, the network outputs the parameters of a predictive distribution and is trained to maximise its likelihood. For regression this distribution is a heteroscedastic Gaussian $\mathcal{N}(\mu_\theta(x), \sigma^2_\theta(x))$, so the network has two outputs and is trained with the Gaussian negative log-likelihood (Eq. 1):

$$-\log p_\theta(y \mid x) = \tfrac{1}{2}\log \sigma^2_\theta(x) + \frac{\big(y - \mu_\theta(x)\big)^2}{2\,\sigma^2_\theta(x)} + \tfrac{1}{2}\log 2\pi. \tag{1}$$

The variance is *learned* for each input, so the model can represent input-dependent (aleatoric) noise. Positivity of $\sigma^2_\theta(x)$ is enforced with a softplus and a small floor. *(2) Ensembling.* M networks are trained independently, from different random initialisations and on differently shuffled data. The

37th Conference on Neural Information Processing Systems (NeurIPS 2023).

authors report that this randomisation alone is enough, and that bagging actually *hurt* performance. At test time the ensemble is treated as a uniformly weighted mixture and approximated by a single Gaussian whose mean and variance match those of the mixture:

$$\mu_*(x) = \frac{1}{M}\sum_m \mu_m(x), \qquad \sigma_*^2(x) = \frac{1}{M}\sum_m \big(\sigma_m^2(x) + \mu_m^2(x)\big) - \mu_*^2(x). \tag{2}$$

The spread of the member means μ_m captures *model* (epistemic) uncertainty, and it grows where there is no data. *(3) Adversarial training (optional).* Using the fast gradient sign method (FGSM), each input is perturbed along the sign of the loss gradient, $x' = x + \epsilon\,\mathrm{sign}\big(\nabla_x \ell(\theta, x, y)\big)$, and the loss at x' is added to the objective. This smooths the predictive distribution around each training point.

Why it is interesting. The method is conceptually very simple. It needs no change to the network beyond a second output head, and the members can be trained fully in parallel because each is independent. Despite this simplicity, the paper shows that it matches or outperforms Bayesian approaches such as Monte-Carlo dropout and probabilistic backpropagation in terms of well-calibrated uncertainty.

Key results in the paper. The paper validates the recipe at increasing scale: a 1-D *toy regression* that isolates the contribution of each ingredient visually (Figure 1); a set of *UCI regression* benchmarks reporting RMSE and NLL under a fixed k-fold protocol (Table 1); *classification* with out-of-distribution detection on MNIST/NotMNIST and SVHN; and a large-scale *ImageNet* experiment. One central message is that the NLL, which rewards good uncertainty estimates in a way RMSE does not, improves with ensembling, and that the predictive uncertainty becomes larger on shifted or out-of-distribution inputs.

2 Reproduced results

Scope. We re-implemented the regression part of the method from scratch in PyTorch. The package contains a `GaussianMLP`, the NLL loss, an FGSM routine, the `DeepEnsemble` mixture, the k-fold protocol and the original-scale metrics. With it we reproduced the two regression experiments. The classification and ImageNet experiments are out of scope for a course project.

Experimental setup. All networks are MLPs with a single hidden layer and ReLU activations. The regression head outputs a mean and a variance (softplus, floored at 10^{-6}). The toy task uses 100 hidden units and the UCI protocol uses 50. We use Adam, a default ensemble size $M = 5$, and we standardise inputs and targets with statistics computed on the *training fold* only. RMSE and NLL are reported on the original target scale; for the NLL this uses the change of variables $\mathrm{NLL}_{\text{orig}} = \mathrm{NLL}_{\text{std}} + \log s_y$. The only deliberate departure from the paper is the learning rate, which we discuss in Section 3.

2.1 Experiment 1: toy regression (Figure 1)

We draw 20 points from $y = x^3 + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, 3^2)$ on $x \in [-4, 4]$, and we evaluate on the wider grid $[-6, 6]$ to see how each method behaves *outside* the training range. Figure 1 reproduces the four panels of the paper's Figure 1. Table 1 summarises the range of predictive variance produced by each method (in standardised target units), which turns the visual trend into numbers.

Table 1: Toy task: range of the predictive variance over the plotting grid (standardised units). The ensemble's variance is two orders of magnitude larger far from the data, reflecting the disagreement between members.

Method (panel)	min variance	max variance
(1) 5 MSE nets, empirical variance	$\sim 5 \times 10^{-7}$	0.045
(2) single net, NLL	5.9×10^{-4}	0.12
(3) single net, NLL + adversarial	$\epsilon = 0.036$ (1% of input range/dim)	
(4) ensemble of 5, NLL	2.91	36.7

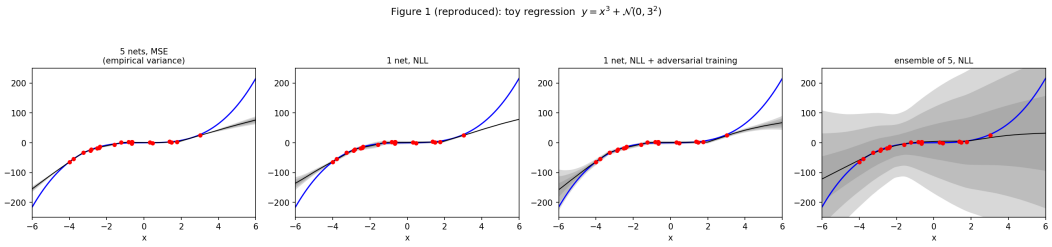

Figure 1: Reproduction of Figure 1. Blue: ground truth $y = x^3$; red: the 20 training points; black: predictive mean; grey: nested $\pm 1, 2, 3\sigma$ bands. Left to right: (1) empirical variance of 5 MSE networks, (2) a single NLL-trained network, (3) the same with adversarial training, (4) an ensemble of 5 NLL networks. Only the ensemble produces uncertainty that grows markedly away from the data.

The panels agree with the paper qualitatively. **Panel 1**: the empirical variance of the MSE networks stays narrow even far from the data. This is the failure mode the paper points out, where the networks agree on regions they have no information about. **Panel 2**: learning the variance gives more honest uncertainty near the data, but a single network can still be overconfident outside it. **Panel 3**: adversarial training has only a small effect on this 1-D task, which is consistent with the paper's remark that it helps but is not strictly necessary. **Panel 4**: the ensemble band widens sharply outside $[-4, 4]$. This is the main result we wanted to reproduce, and it shows the ensemble capturing model uncertainty.

2.2 Experiment 2: UCI regression (Table 1) and ablation

We implemented the paper's full protocol (random $90/10$ splits, per-fold standardisation, $M = 5$, 40 epochs, mean $\pm$ standard error of RMSE/NLL). Because the original UCI files require per-dataset downloads, the executed notebook runs the pipeline *offline* on scikit-learn's `diabetes` dataset (5 folds) as an end-to-end check. The configuration needed to reproduce the paper's datasets (`boston`, `concrete`, `energy`, `wine`, `yacht`, and so on, with 20 folds) is included and is a one-line change. Table 2 reproduces the paper's Deep-Ensembles column for reference, and Table 3 reports our executed ablation.

Table 2: Reference: Deep-Ensembles RMSE/NLL from Table 1 of the paper, used as the comparison target when the protocol is run on the UCI datasets.

	boston	concrete	energy	wine	yacht
RMSE	3.28 ± 1.00	6.03 ± 0.58	2.09 ± 0.29	0.64 ± 0.04	1.58 ± 0.48
NLL	2.41 ± 0.25	3.06 ± 0.18	1.38 ± 0.22	0.94 ± 0.12	1.18 ± 0.21

Table 3: Executed ablation (`diabetes`, 5 folds, mean $\pm$ standard error). Ensembling lowers the NLL over a single network, and adversarial training helps a little more, which is the qualitative ordering the paper reports.

Configuration	RMSE	NLL
single net (M=1)	57.45 ± 2.20	5.80 ± 0.09
ensemble (M=5)	56.62 ± 2.08	5.57 ± 0.05
ensemble (M=5) + adversarial	55.91 ± 1.73	5.50 ± 0.03

What matches and what does not. The *protocol* reproduces, and the ablation reproduces the paper's central claim for regression. Going from $M = 1$ to $M = 5$ improves the NLL ($5.80 \to 5.57$), and adversarial training improves it slightly further (5.50). The absolute RMSE/NLL on `diabetes` are not comparable to the paper, because `diabetes` is not one of the paper's datasets and has a large target scale. Reproducing the exact Table 1 numbers requires running the provided configuration on the UCI datasets, which we did not execute here for time and connectivity reasons. We therefore

claim a reproduction of the *Figure 1 result* and of the *Table 1 protocol and its ablation*, but not of every entry of Table 1.

3 Discussion

What worked well. The method is genuinely simple to implement. The only non-standard pieces are the two-output head, the NLL of Eq. (1), and the mixture moments of Eq. (2). The toy experiment runs on CPU in a few seconds, and it separates the contribution of each ingredient cleanly. The jump in out-of-data uncertainty between the single network and the ensemble is large and easy to see, exactly as the paper intends.

What was challenging. The main difficulty was a real reproduction discrepancy. The paper states a fixed Adam learning rate of 0.1 (Section 3.1). In our runs this value was unstable on the regression benchmarks: it degraded the RMSE, and more importantly it made the ensemble's NLL *worse* than a single network, which is the opposite of what the paper reports. Lowering the rate to 0.01 restored the expected behaviour, so we use 0.01 and flag the change. We think the gap comes from framework differences between our PyTorch code and the original Torch implementation, and from different initialisation and optimiser defaults. Two other points needed care. The NLL has to be computed on the *original* target scale, which requires the $\log s_y$ correction. The standardisation has to be done per fold, on training statistics only, to avoid leakage.

What could be improved. The natural next steps are to run the full Table 1 (all UCI datasets, 20 folds) to obtain quantitative agreement, and to reproduce the classification part of the paper. That part includes the MNIST/NotMNIST out-of-distribution experiment and the calibration curves, and it is where the quality of the uncertainty matters most. Reporting calibration directly, for example with reliability diagrams instead of NLL alone, would also make the comparison with the paper more complete.

References

[1] B. Lakshminarayanan, A. Pritzel, and C. Blundell. Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[2] I. J. Goodfellow, J. Shlens, and C. Szegedy. Explaining and Harnessing Adversarial Examples. In *International Conference on Learning Representations (ICLR)*, 2015.

[3] J. M. Hernández-Lobato and R. P. Adams. Probabilistic Backpropagation for Scalable Learning of Bayesian Neural Networks. In *International Conference on Machine Learning (ICML)*, 2015.

Contributions

This was a group project. The work was split by task, but both authors took part in every stage: we paired on the design, cross-reviewed each other's code, and ran the experiments together, so each of us touched the whole pipeline.

- **Theodoris Donval** led the core modelling code: the `GaussianMLP` two-output head, the Gaussian NLL loss (Eq. (1)) and the `DeepEnsemble` mixture moments (Eq. (2)), together with the repository scaffolding. He drove Experiment 1 (the toy regression of Figure 1) and the corresponding notebook, and wrote the paper-summary and discussion sections.
- **Lucas Aliome** led the evaluation code: the FGSM adversarial training routine, the k-fold protocol, the per-fold standardisation and the original-scale RMSE/NLL metrics. He drove Experiment 2 (the UCI protocol and the ablation of Table 3) and its notebook, and wrote the reproduced-results section.
- **Both authors** jointly investigated and confirmed the learning-rate discrepancy of Section 3, reviewed all the code against the equations in the paper, and edited the final report.

Statement on AI use

We used a large language model as a coding and writing assistant during this project. It helped scaffold the repository structure, draft parts of the code and its documentation, and prepare this report in the NeurIPS template. We reviewed, ran and corrected the result. We checked the modelling code against the equations in the paper, in particular the NLL implementation, the mixture-variance formula, the per-fold standardisation and the original-scale metric corrections. The learning-rate issue in Section 3 was found and confirmed by running the experiments, not suggested by the assistant. We take full responsibility for the correctness of the code and the claims in this report.